Measures of Query Cost

Core Concept Constant: Query optimization selects the most efficient plan by estimating the **cost** of individual operations. While early models focused solely on disk I/O, modern systems must account for CPU and memory hierarchies.

1. Components of Query Cost

The cost of executing a query is driven by the following resource usages:

1. **Disk Access Cost:** Number of blocks read/written from HDD or SSD storage.
2. **CPU Cost:** Time spent processing tuples, evaluating predicates, and sorting.
3. **Network Communication Cost:** Dominant in parallel and distributed systems.

Storage Environments

- **HDD (In-storage):** Cost dominated by physical seek and transfer times.
- **SSD (Flash):** Zero physical seek latency, but interface and controller latency persist.
- **In-Memory Database:** Query cost is governed by CPU time and cache-line memory access.

2. PostgreSQL Cost Model (2018)

Modern PostgreSQL versions estimate cost using a combination of:

- **CPU cost per tuple:** Overhead of processing a row.
- **CPU cost per index entry:** Traversing and managing index nodes.
- **CPU cost per operator:** Evaluating arithmetic or comparison filters.

3. Choice of Metric: Resource vs. Time

Total Resource Consumption is favored over **Response Time** because:

- Response time is highly unpredictable (depends on concurrent load).
- In shared databases, minimizing resource use allows higher query throughput.

Example: A parallel plan might have a lower response time but a higher total cost due to redundant work. Minimizing total work is usually the safer heuristic.

4. Detailed I/O Cost Estimation

For disk-based operations, cost is expressed as blocks transferred (b) and seeks (S):

$$\boxed{\text{Total Cost} = b \times t_T + S \times t_S}$$

Storage Parameters (Typical for 4 KB blocks)

Storage Type	**Seek Time (t_S)**	**Transfer Time (t_T)**
High-end Magnetic Disk (HDD)	4 ms	0.1 ms
SATA SSD	$\approx 90\,\mu$s	$\approx 10\,\mu$s
PCIe 3.0×4 SSD	$20 - 60\ \mu$s	$\approx 2\,\mu$s
Main Memory	< 100 ns	$< 1\,\mu$s

Read vs. Write Cost

- **Verify Step:** Writes are typically slower than reads as the system often reads back data to verify the write operation.
- **HDD/SSD Rule:** Writes generally cost approximately **twice** as much as reads ($2 \times t_T$).

5. Example: HDD vs. SSD Comparison

Workload: 1,000 Blocks + 50 Seeks

HDD ($t_S = 4ms, t_T = 0.1ms$):

$$1,000(0.1) + 50(4) = 100 + 200 = \mathbf{300}\,\text{ms}$$

SATA SSD ($t_S = 90\mu s, t_T = 10\mu s$):

$$1,000(0.01) + 50(0.09) = 10 + 4.5 = \mathbf{14.5}\,\text{ms}$$

6. Buffer and Buffer Cache Model

Query performance depends heavily on whether data is already resident in memory.

PostgreSQL Cache Assumptions

- **Effective Cache Size:** Default assumption is 4 GB of RAM available for caching.
- **Hit Ratio Heuristic:** Systems may assume a 90% cache hit ratio, reducing the "effective" cost of random reads by a factor of 10.
- **B^+-tree Nodes:** High-level internal nodes are assumed to be permanently cached; only leaf-level I/O is typically counted.

Estimation Scenarios

- **Worst Case:** Assume an empty cache and minimum memory availability.
- **Optimistic Case:** Assume useful data remains from previous queries.

7. Summary

- Query optimization targets **Total Resource Consumption**.
- Disk cost scales linearly with block transfers (b) and latency (S).
- CPU factors (tuple processing, ops) are crucial for SSD/In-memory performance.
- Buffer residency is the primary uncertainty in cost modeling.